Doesn’t that really help us predicting the performance of either algorithm on another problem instance?

Conversely, if we can prove that, on an array with `n` elements, algorithm A needs `n * n` element swaps, while algorithm B only takes `n`, we have a very good way to predict how each of them will perform on input of any size.

This is where big-O notation kicks in. But first, take a look at figure B.1 to get an idea of how the running times for these two algorithms would compare: this chart should make you realize why we would like to stick with algorithm B.

B.4 Notation

Big-O notation is a mathematical symbolization used to describe how certain quantities grow with the size of an input. Usually it takes the form of a capital `O`, enclosing an expression within parenthesis: something like `O(f(n))`, hence the name big-O. `f(n)` here can be any function with input `n`; we only consider functions on integers, as `n` usually describe the size of some input.

We are not going to describe big-O notation in details here, it's way out of our scope, but there are many textbooks available with an in-depth dissertation on the topic.

The main concept you need to remember is that the notation `O(f(n))` expresses a bound.

Mathematically, saying `g(n) = O(f(n))` means that for any large enough input, there is a Real value constant `c` (whose